

KijaniKiosk Capstone — Project Scope Document

Track A: Infrastructure-First

Problem Statement

KijaniKiosk's kk-payments service has no isolated staging environment: every change is deployed directly to production with no automated health check, no human approval step, and no alerting on failure. A single bad deploy can reach customers with no gate and no early warning. This project builds a staging environment, an automated deploy pipeline with a smoke test and approval gate, and a restart-count alert, so that a bad change is caught in staging and requires a documented human decision before it reaches production.

In Scope

- A kijani-staging Kubernetes namespace, provisioned by Terraform, isolated from the default production namespace.
- kk-payments deployed to both staging and production from one shared Deployment manifest, with environment-specific ConfigMaps (different DB_HOST per environment).
- A Jenkins pipeline that deploys to staging, runs an automated smoke test against /healthz, and only offers a production approval gate after the smoke test passes.
- A Prometheus alert rule that fires when kk-payments restarts more than 3 times in a 15-minute window.
- RBAC scoping the Jenkins build agent's permissions to only the kijani-staging and default namespaces.

Out of Scope

- A real kk-payments application container. No production-ready image exists in this environment; hashicorp/http-echo is used as a placeholder that satisfies the same probes, ports, and ConfigMap wiring a real container would. Reason: building the actual payments application was outside the capstone's infrastructure scope, and the placeholder proves the Deployment/pipeline/RBAC logic independent of application code.
- The Week 10 serverless receipt chain (kk-payments-receipts-staging bucket, Lambda-equivalent functions). Reason: the original Week 10 project files were not available when this capstone was built, and this environment has no cloud provider credentials configured, so there was no real target to deploy serverless functions to. The ConfigMaps already define RECEIPTS_BUCKET so the integration seam is prepared for future work.

Success Criteria (each demonstrable live)

- A push to main triggers the Jenkins pipeline (via Poll SCM) with no manual step, and it deploys kk-payments to kijani-staging automatically.
- The pipeline's smoke test against staging's /healthz endpoint returns HTTP 200 and is shown passing in the console output.
- The pipeline pauses at an approval gate, accepts a typed APPROVAL_REASON, and only then deploys to production — shown live with a real reason entered.
- kubectl get configmap ... -o jsonpath='{.data.DB_HOST}' run against both namespaces returns two different values, proving environment separation from one shared Deployment manifest.
- The Prometheus alert rule is visible and loaded (state: inactive/armed) at /alerts in the Prometheus UI, with its query and threshold shown.

Architecture

See architecture-diagram.png. Developer pushes to GitHub main → Jenkins (Poll SCM) checks out the repo and installs kubectl → deploys kk-payments to kijani-staging via kubectl apply -k → runs a smoke test against the staging Service over a port-forward → pauses at a human approval gate → on approval, deploys the same manifest to the default (production) namespace. Prometheus scrapes kk-payments pods in both namespaces and evaluates a restart-count alert rule. Jenkins authenticates to the Kubernetes API using its own build agent's in-cluster ServiceAccount, scoped by RBAC to only the two target namespaces.